

Clarity

Instructor: Zeeshan Nazar

Section: BCS 5J

Team members

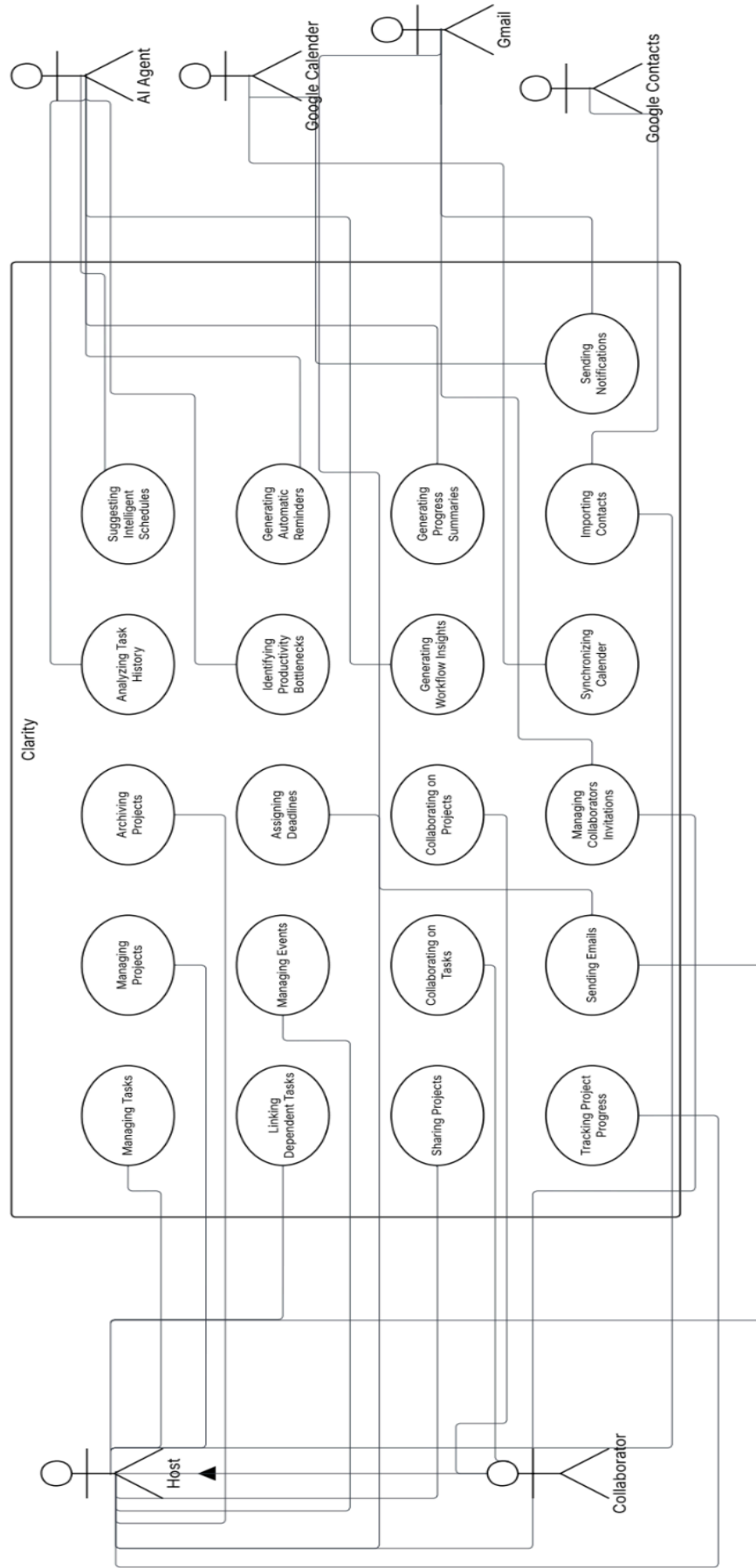
23L-0613 Mawahid Abbas

23L-0757 Bilal Kashif

23L-0848 Mohammad Hamza Iqbal (Team Leader)

Table of contents

Use case diagram.....	2
UC 01-managing tasks.....	3
UC 02-linking independent tasks.....	4
UC 03-sharing projects.....	5
UC 04-tracking project progress.....	6
UC 05-managing projects.....	7
UC 06-managing events.....	8
UC 07-collaborating on tasks.....	9
UC 08-sending emails.....	10
UC 09-archiving projects.....	11
UC 10-assigning deadlines.....	12
UC 11-collaborating on projects.....	13
UC 12-managing collaborator invitations.....	14
UC 13-analysing task history.....	15
UC 14-identifying productivity bottlenecks.....	16
UC 15-generating workflow insights.....	17
UC 16-synchronizing calendar.....	18
UC 17-suggesting intelligent schedules.....	20
UC 18-generating automatic reminders.....	21
UC 19-generating project summaries.....	22
UC 20-importing contacts	23
UC 21-sending notifications.....	24
Analysis Class Diagram.....	25



Identifier	UC-01	
Name	Managing Tasks	
Summary	This use case describes how the host manages tasks within the system, including creating, editing, deleting, and organizing tasks.	
Priority	High	
Actors	Host	
Pre-condition(s)	Host must be logged into the system. A project or workspace must already exist.	
Post-condition(s)	Tasks are successfully updated in the system. All participants can view the updated task list in real time.	
Dependencies	Depends on Managing Projects i.e, a project or workspace must exist.	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects "Manage Tasks" option.	System displays the task management interface.
2	Host creates a new task by entering details (title, description, deadline).	System verifies and saves the task.
3	Host edits an existing task.	System updates the task details.
4	Host deletes a task.	System removes the task and updates the list.
5	Host organizes tasks by drag-and-drop into categories or priority.	System updates the task order and synchronizes changes in real time.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Host enters incomplete or invalid task details.	System prompts host to correct the details.
3a	Host attempts to edit a non-existent or deleted task.	System displays an error message.

4a	Host tries to delete a locked or assigned task.	System displays a warning and prevents deletion.
-----------	---	--

Exceptions:

System fails to save or update tasks due to server issues.

Identifier	UC-02	
Name	Linking Dependent Tasks	
Summary	The host links tasks together to establish dependencies (e.g., one task must be completed before another begins).	
Priority	Medium	
Actors	Host	
Pre-condition(s)	Tasks must already exist in the project.	
Post-condition(s)	Dependency relationships are created and visible in task flow.	
Dependencies	Depends on Managing Tasks as tasks must exist first.	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects tasks to be linked.	System highlights selected tasks.
2	Host sets dependency relationship (e.g., Task B depends on Task A).	System records and displays dependency.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Host attempts to link a task to itself.	System shows error.
2b	Host tries to link circular dependencies (A→B→A).	System blocks and alerts host.

Exceptions:

Dependency not saved due to server error.

Identifier	UC-03	
Name	Sharing Projects	
Summary	The host shares a project with collaborators for joint work.	
Priority	High	
Actors	Host	
Pre-condition(s)	Project must exist. Collaborators must have accounts.	
Post-condition(s)	Project is shared, and collaborators gain access.	
Dependencies	Depends on Managing Projects as a project must exist.	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects "Share Project."	System displays sharing options.
2	Host enters collaborator's email/ID.	System validates collaborator.
3	Host sets access level (view/edit).	System applies permissions.
4	Host confirms sharing.	System sends invitation to collaborator.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Invalid email ID entered.	System displays error.
3a	Host tries to assign unsupported permission.	System shows warning.

Exceptions:

Email server or notification service unavailable.

Identifier	UC-04	
Name	Tracking Project Progress	
Summary	The host tracks the project’s progress through visual dashboards, analytics, and reports.	
Priority	Medium	
Actors	Host	
Pre-condition(s)	Project and tasks must exist.	
Post-condition(s)	Progress data is displayed.	
Dependencies	Depends on Managing Tasks and Linking Dependent Tasks as tasks and dependencies provide progress data.	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects “View Progress.”	System loads dashboard.
2	Host selects metrics (e.g., completion rate, deadlines).	System generates visual reports.
Alternate Course of Action		
S#	Actor Action	System Response
2a	No tasks exist.	System shows empty progress message.

Exceptions:

Analytics service fails.

Identifier	UC-05	
Name	Managing Projects	
Summary	The host creates, edits, or deletes entire projects.	
Priority	High	
Actors	Host	
Pre-condition(s)	Host must be logged in.	
Post-condition(s)	Project list updated.	
Dependencies	No dependency	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects "Manage Projects."	System displays projects list.
2	Host creates/edits/deletes project.	System updates project records.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Host tries to delete project with active collaborators.	System asks for confirmation.

Exceptions:

Project not saved or deleted due to system error.

Identifier	UC-06	
Name	Managing Events	
Summary	The host manages events within a project (e.g., deadlines, meetings).	
Priority	Medium	
Actors	Host	
Pre-condition(s)	Project must exist.	
Post-condition(s)	Events updated and synchronized.	
Dependencies	Depends on Managing Projects as events are tied to projects.	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects "Manage Events."	System shows calendar view.
2	Host creates, edits or deletes event.	System updates calendar.
Alternate Course of Action		
S#	Actor Action	System Response
2a	User enters an invalid date.	System prompts correction.
2b	User enters an invalid time.	System prompts correction.

Exceptions:

Integration with calendar API fails.

Identifier	UC-07	
Name	Collaborating on Tasks	
Summary	Collaborators work jointly on tasks, adding comments, making edits, and tracking updates.	
Priority	High	
Actors	Collaborator	
Pre-condition(s)	Project must be shared with collaborator.	
Post-condition(s)	Task collaboration history updated.	
Dependencies	Depends on Sharing Projects and Managing Tasks as project must be shared and tasks must exist.	
Typical Course of Action		
S#	Actor Action	System Response
1	Collaborator opens shared task.	System displays task details.
2	Collaborator adds comments or edits task.	System updates and synchronizes changes.
3	Collaborator checks activity log.	System shows updates from all collaborators.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Collaborator tries to edit with read-only access.	System denies and shows warning.

Exceptions:

Synchronization failure due to connection issue.

Identifier	UC-08	
Name	Sending Emails	
Summary	The host sends emails to collaborators through the integrated Gmail service.	
Priority	Medium	
Actors	Primary: Host Supporting: Gmail	
Pre-condition(s)	Host must be logged in. Collaborator’s email address must be available. Gmail integration must be active.	
Post-condition(s)	Email is successfully sent and collaborator receives it.	
Dependencies:	Depends on Sharing Projects (project context for communication) and requires Gmail integration.	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects “Send Email.”	System displays email composition window.
2	Host enters recipient(s), subject, and message.	System validates email details.

3	Host clicks "Send."	System routes message to Gmail API.
4	Gmail sends the email.	System confirms successful delivery.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Invalid email format entered.	System shows error.
3a	Email exceeds attachment size.	System notifies host.

Exceptions:

Gmail API unavailable.

Internet connectivity lost.

Identifier	UC-09	
Name	Archiving Projects	
Summary	The host archives completed projects to keep the workspace organized.	
Priority	Medium	
Actors	Host	
Pre-condition(s)	Project must exist.	
Post-condition(s)	Project is marked as archived and hidden from active workspace.	
Dependencies	Depends on Managing Projects (a project must already exist).	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects "Archive Project."	System confirms action.
2	Host approves archive request.	System archives project and moves it to archive folder.
Alternate Course of Action		

S#	Actor Action	System Response
1a	Host cancels request.	Project remains active.

Exceptions:

Archive action fails due to server error.

Identifier	UC-10	
Name	Assigning Deadlines	
Summary	The host assigns deadlines to tasks or projects.	
Priority	High	
Actors	Host	
Pre-condition(s)	Task or project must exist.	
Post-condition(s)	Deadline assigned and reflected in project calendar.	
Dependencies	Depends on Managing Tasks (tasks must exist before deadlines can be assigned).	
Typical Course of Action		
S#	Actor Action	System Response
1	Host selects task/project.	System displays details.
2	Host sets deadline date or time.	System validates entry.

3	Host saves changes.	System updates and displays deadline in calendar.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Invalid date or time entered (past deadline).	System prompts correction.

Exceptions:

Calendar service unavailable.

Identifier	UC-11	
Name	Collaborating on Projects	
Summary	Collaborators work jointly on shared projects, including editing details, assigning tasks, and tracking updates.	
Priority	High	
Actors	Collaborator	
Pre-condition(s)	Project must be shared with collaborator.	
Post-condition(s)	Collaboration changes are reflected in real time.	
Dependencies	Depends on Sharing Projects (project must be shared first).	
Typical Course of Action		
S#	Actor Action	System Response
1	Collaborator opens shared project.	System displays project details.

2	Collaborator edits or updates project details.	System synchronizes updates.
3	Collaborator views changes from others.	System refreshes project data.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Collaborator has read-only access.	System blocks editing.

Exceptions:

Sync failure due to poor connectivity.

Identifier	UC-12
Name	Managing Collaborator Invitations
Summary	The host invites collaborators to projects via Gmail integration.
Priority	High
Actors	Primary: Host Supporting: Gmail
Pre-condition(s)	Project must exist. Collaborator must have an email address.
Post-condition(s)	Invitation is sent and collaborator notified.
Dependencies	Depends on Managing Projects (project exists to invite into) and requires Gmail integration.

Typical Course of Action		
S#	Actor Action	System Response
1	Host selects "Invite Collaborator."	System displays invitation form.
2	Host enters collaborator's email.	System validates email.
3	Host sends invitation.	System routes request to Gmail API.
4	Gmail sends invitation email.	System confirms delivery.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Invalid email entered.	System shows error.
3a	Host cancels request.	System does not send invitation.

Exceptions:

Gmail API fails to send invitation.

Identifier	UC-13
Name	Analyzing Task History
Summary	The AI agent analyzes past task data to generate insights.
Priority	Medium
Actors	AI Agent
Pre-condition(s)	Task history must exist.
Post-condition(s)	AI-generated analysis provided to host.
Dependencies	Depends on Managing Tasks as tasks history must exist.

Typical Course of Action		
S#	Actor Action	System Response
1	AI agent scans completed task history.	System retrieves stored task data.
2	AI agent applies analysis algorithms.	System generates insights and displays them.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Insufficient data provided.	AI informs host that no meaningful analysis can be generated.

Exceptions:

AI service fails.

Identifier	UC-14
Name	Identifying Productivity Bottlenecks
Summary	The AI agent identifies tasks or workflows that cause delays and reduces productivity.
Priority	Medium
Actors	AI Agent
Pre-condition(s)	Task history and progress data must exist.

Post-condition(s)		Host receives suggestions for resolving bottlenecks.
Dependencies		Depends on Analyzing History of Tasks as analysis is prerequisite for identifying bottlenecks.
Typical Course of Action		
S#	Actor Action	System Response
1	AI agent reviews task timelines and completion data.	System loads relevant project data.
2	AI agent identifies overdue tasks or frequent blockers.	System highlights bottlenecks.
3	AI agent suggests improvements.	System presents recommendations to host.
Alternate Course of Action		
S#	Actor Action	System Response
2a	AI finds no bottleneck.	System displays "No issues detected."

Exceptions:

AI analysis fails due to missing data.

Identifier	UC-15
Name	Generating Workflow Insights
Summary	The AI agent analyzes ongoing tasks and projects to generate insights into workflow efficiency.
Priority	Medium

Actors		AI Agent
Pre-condition(s)		Task and project data must exist.
Post-condition(s)		Workflow insights are displayed to the host.
Dependencies		Depends on Analyzing History of Tasks and Identifying Productivity Bottlenecks.
Typical Course of Action		
S#	Actor Action	System Response
1	AI agent scans active projects and tasks.	System retrieves current data.
2	AI agent applies workflow analysis algorithms.	System generates insights.
3	AI agent presents insights.	System displays them in dashboard.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Limited task data.	System shows "insufficient data for insights."

Exceptions:

AI service error.

Identifier	UC-16
Name	Synchronizing Calendar

Summary		The system synchronizes events and deadlines with Google Calendar.
Priority		High
Actors		Google Calender
Pre-condition(s)		Calendar integration enabled.
Post-condition(s)		Events synced across both systems.
Dependencies		Depends on Managing Events i.e, events must exist to sync.
Typical Course of Action		
S#	Actor Action	System Response
1	System prepares event data.	Google Calendar API receives data.
2	Google Calendar confirms sync.	System updates status to host.
Alternate Course of Action		
S#	Actor Action	System Response
1a	Duplicate event detected.	System prevents duplicate entry.

Exceptions:

Google Calendar API unavailable.

Identifier	UC-17	
Name	Suggesting Intelligent Schedules	
Summary	The AI agent suggests optimized schedules for tasks and events based on workload and availability.	
Priority	Medium	
Actors	AI Agent	
Pre-condition(s)	Host must have active tasks/events.	
Post-condition(s)	Suggested schedule displayed.	
Dependencies	Depends on Synchronizing Calendar and Assigning Deadlines.	
Typical Course of Action		
S#	Actor Action	System Response
1	AI agent reviews deadlines and availability.	System retrieves relevant data.
2	AI agent generates intelligent schedule.	System displays recommendations.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Conflicting deadlines detected.	System highlights conflicts for manual resolution.

Exceptions:

AI scheduling module fails.

Identifier	UC-18	
Name	Generating Automatic Reminders	
Summary	The AI agent generates reminders for tasks, events, or deadlines.	
Priority	High	
Actors	AI Agent	
Pre-condition(s)	Tasks or events must exist with deadlines.	
Post-condition(s)	Reminder notifications sent to host.	
Dependencies	Depends on Suggesting Intelligent Schedules and Managing Events.	
Typical Course of Action		
S#	Actor Action	System Response
1	AI agent scans upcoming deadlines.	System identifies tasks/events.
2	AI agent generates reminder.	System schedules reminder notification.
Alternate Course of Action		
S#	Actor Action	System Response
2a	No upcoming deadlines.	System displays “no reminders.”

Exceptions:

Notification service fails.

Identifier		UC-19
Name		Generating Progress Summaries
Summary		The AI agent generates summaries of project progress for host review.
Priority		Medium
Actors		AI Agent
Pre-condition(s)		Project or task data must exist.
Post-condition(s)		Progress summary displayed in dashboard.
Dependencies		Depends on Tracking Project Progress and Analyzing History of Tasks.
Typical Course of Action		
S#	Actor Action	System Response
1	AI agent reviews task completion data.	System aggregates data.
2	AI agent generates summary.	System displays progress summary.
Alternate Course of Action		
S#	Actor Action	System Response
1a	No completed tasks.	System shows "progress summary unavailable."

Exceptions:

AI service failure.

Identifier	UC-20	
Name	Importing Contacts	
Summary	The system imports user contacts from Google Contacts for collaboration and communication.	
Priority	Medium	
Actors	Google Contacts	
Pre-condition(s)	Google Contacts integration enabled.	
Post-condition(s)	Contacts successfully imported.	
Dependencies	No direct dependency except system login; may indirectly support Managing Collaborator Invitations.	
Typical Course of Action		
S#	Actor Action	System Response
1	System requests contacts.	Google Contacts API provides list.
2	System imports contacts.	System updates user’s contact directory.
Alternate Course of Action		
S#	Actor Action	System Response
2a	Duplicate contact found.	System merges or skips.

Exceptions:

Google Contacts API unavailable.

Identifier	UC-21	
Name	Sending Notifications	
Summary	The system sends notifications (e.g., reminders, updates) to users using Gmail and Google Calendar.	
Priority	High	
Actors	Google Calendar Gmail	
Pre-condition(s)	Notifications must be scheduled.	
Post-condition(s)	User receives notification successfully.	
Dependencies	Depends on Managing Events, Sharing Projects, and external services Google Calendar and Gmail.	
Typical Course of Action		
S#	Actor Action	System Response
1	System generates notification event.	Google Calendar/Gmail API processes request.
2	Notification sent to user.	System confirms delivery status.
Alternate Course of Action		
S#	Actor Action	System Response
1a	Invalid recipient.	System shows error.

Exceptions:

Gmail API failure.

Google Calendar API failure.

